

Guia do paper (para o Rafa)

Este documento não vai junto na entrega. Ele existe para você **entender e defender** cada parte do paper na conversa com a AdzHub. Lê uma vez antes da entrevista e você fala de tudo com segurança.

A tese em uma frase

O erro comum é tratar tudo como "tool". Eu proponho um harness **híbrido** onde cada uma das três camadas do domínio ganha um mecanismo diferente: a **memória** (supercérebro) é hidratada como **ambiente** antes do loop, as **APIs** (Meta, CRM) são **tools** num loop ReAct, e os **Apps** de metodologia são **skills** encapsuladas.

Se te perguntarem "resuma sua tese", é isso. O nome é **Harness Tri-Camada**.

Por que isso importa (a intuição)

O gestor de marketing não faz perguntas soltas. Toda pergunta dele pressupõe o **histórico da conta** ("o CPA subiu", "por que os leads dizem Google"). Se você trata a memória como só mais uma tool que o LLM chama quando lembra, dois problemas aparecem:

1. O LLM esquece de buscar contexto e responde no vácuo.
2. Ou busca demais, gastando passos e tokens.

A sacada é: **memória não é uma pergunta que o agente faz, é o chão onde ele pisa**. Então o harness recupera o contexto relevante e injeta ANTES do loop começar. O agente já nasce situado. Isso é o que a literatura chama de "contexto como ambiente" (RLM).

Ao mesmo tempo, os **Apps** (análise de criativos, mapa de solução) são metodologia pronta da AdzHub. Se o agente reimplementasse a metodologia no prompt, você teria duas versões divergindo. Então o harness **chama o app** e usa a saída. Isso é "skill".

E as **APIs** (Meta, CRM) são dado cru, sem estado. Essas sim são tools clássicas que o LLM decide chamar. O trabalho inteligente ("cruzar gasto do Meta com venda do CRM por utm_content") é do agente, não da tool.

A analogia do cérebro e do exoesqueleto (a sua, e está no paper)

- O **LLM** é um cérebro num pote: sabe muito, mas não vê o mundo nem age.
- O **harness** é o exoesqueleto: restringe o foco, dá músculos e ferramentas (tools, memória, dados) e força a saída num comportamento estrito.
- **Não muda o cérebro**. Não mexemos nos pesos do modelo (isso seria treino). Só embrulhamos software em volta e filtramos o que ele vê.

Serve para explicar em 20 segundos a diferença entre **modelo** e **harness**, que é a primeira coisa que o paper separa.

Passeio pelo paper, seção por seção

Abstract (Resumo)

O que diz: o problema do gestor, a tese (híbrido de 3 mecanismos), o que o protótipo ilustra e o recorte consciente (só-leitura). **Por que está aí:** um avaliador lê o abstract e decide se entendeu sua tese em 15 linhas. É o elevador. **Como defender:** "Ataco tarefas compostas do gestor (cruzar Meta x CRM, diagnosticar CPA, montar pauta) tratando cada camada do domínio com o mecanismo certo."

1. Introdução

O que diz: separa **modelo** (o LLM, prevê token) de **harness** (o runtime: loop, tools, limites, estado). Traz a analogia do exoesqueleto. Fecha com a frase de tese. **Por que está aí:** responde a pergunta 1 (tese), a 2 (o que deixa de ser chatbot) e a 3 (o que mudou na minha leitura depois de estudar). **Como defender:** "No começo eu achava que harness era só o loop de tool-calling. Estudando, entendi que a decisão de arquitetura não é 'qual loop', é **como cada recurso do domínio entra no runtime**: o que vira tool, o que vira memória, o que vira skill."

2. Preliminares

2.1 (As peças do exoesqueleto) nomeia as seis partes de um harness pelo que elas **fazem no trabalho do gestor**, não por jargão: o ciclo de passos, as mãos (tools), o tato (observação), a memória, as travas (allowlist e teto de passos) e o que sobrevive entre as voltas (estado). **2.2** lê o domínio AdzHub em três camadas (supercérebro / Apps / APIs), crava a observação central (essas camadas têm **naturezas diferentes**) e destrincha as quatro decisões que vêm disso: por que o supercérebro é **grafo** e não tabela (o que importa está na ligação, não no registro), por que a linha do tempo é **camada** e não campo de data (fato de operação vence, e memória que só acumula responde com o fato revogado), por que um **App não é só mais uma tool** (tool carrega dado, App carrega opinião do negócio; metodologia no prompt custa token, perde consistência e deixa de ser versionável) e o que os **canais** mudam (WhatsApp é onde mora o porquê; Meta, CRM e Analytics contam a mesma venda de três jeitos, e o desenho expõe a divergência em vez de eleger um vencedor em silêncio). **2.3 (Cinco famílias de harness)** é a seção de estudo: o que cada família de fato propõe (ReAct, CodeAct/sandbox, sessão com permissões e skills, orquestração por grafo, contexto como ambiente), e a **Tabela 1** põe as cinco lado a lado com quatro colunas, sendo que a que decide é **"onde falha no chat do gestor"**. É ela que explica por que a resposta não é escolher uma família, e sim compor. **Por que está aí:** o resto do paper precisa dessas palavras. Mas glossário seco não convence ninguém, então cada peça vem colada numa cena real (abrir o Gerenciador, ver o CPA, abrir o CRM). **Como defender:** "O exoesqueleto tem seis peças, e nenhuma delas está no modelo: todas são do runtime em volta. É por isso que dois produtos com o mesmo LLM entregam agentes diferentes." **Se perguntarem o que é 'observação':** é o que a mão traz de volta. É ela que faz o agente mudar de ideia no meio do caminho, quando o CRM mostra 2 vendas onde o Meta prometia 14.

3. Arquitetura (o coração)

3.1 argumenta a **composição**: o veredito família por família já está na Tabela 1 (§2.3), e aqui o texto defende por que cada camada recebe um mecanismo diferente (núcleo ReAct porque a tarefa é aberta, memória fora do loop porque toda pergunta pressupõe histórico, Apps como skills porque metodologia é produto e não prompt). Isso é o que a pergunta 4 pede. A frase que fecha: *a tese não está em nenhuma das três escolhas isoladas, está na composição*. **3.2** responde a pergunta 5 (o que é tool, o que é memória, o que é app), item por item. **3.3** lista os trade-offs aceitos de propósito (pergunta 6). **Figura 1** desenha tudo: gestor → chat → harness (hidrata / loop / skills) → 3 camadas → ação. **Como defender cada rejeição:**

- **ReAct puro:** certo como esqueleto, mas sozinho vira "memória é tool" e o agente responde sem chão. Mantenho o loop, tiro a memória de dentro dele.

- **Sandbox/CodeAct:** o trabalho aqui não é rodar código, é cruzar tabelas. Sandbox adiciona risco e latência sem resolver o problema. Fica fora do MVP.
- **Permissões & skills:** adoto a metade de **skills** (os Apps são skills). Adio **permissões** porque o MVP é só-leitura.
- **Grafo de estados:** ótimo para fluxo fixo. O chat do gestor é aberto, uma tarefa por mensagem. O supercérebro é um grafo de **dados**, não de **controle**. Uso o grafo como memória, não como máquina de estados.
- **RLM (contexto como ambiente):** é a peça que cura a fraqueza do ReAct puro. É de onde vem a "hidratação".

3.4 (Memória entre sessões) é a sua ideia, analisada: ao fim de cada sessão o harness grava um resumo dos pontos principais, e a próxima sessão consulta. **Defenda pelo custo, não pela conveniência:** é a única forma barata de a mensagem 40 não reenviar as 39 anteriores, e ela cabe na tese sem tocar no loop, porque o resumo entra como memória hidratada, não como tool nova. Os **seis riscos** estão no paper e é bom saber dizer os três principais de cabeça: (a) o resumo resume da FONTE, nunca de outro resumo, senão o desvio compõe como telefone sem fio; (b) guarda **decisão e intenção com data, número nunca**, porque número vem da tool na hora da pergunta e um resumo antigo contradiz o dado novo com a mesma confiança; (c) cada ponto carrega ponteiro para o evento que o sustenta, porque quem escreve o resumo é o modelo, e resumo sem revisão é texto não verificado entrando na memória permanente.

3.5 (Falha e paralelismo) responde uma pergunta que o paper não respondia. A frase que resume: **falha vira observação, nunca exceção** — o loop continua e a resposta diz o que faltou, porque um agente que responde igual com e sem metade dos dados é pior que um que não responde. E a distinção que mais vale citar: **dado vazio não é erro, é resposta**; confundir os dois faz o agente concluir que o anúncio não vendeu quando o CRM apenas não respondeu.

4. Sistema (o que o protótipo ilustra)

O que diz: descreve o protótipo (dois motores: simulado e LLM), o trace visível, e um **turno concreto** (intent → hidratação → tools → observação → resposta) para o diagnóstico do Ômega 3. Tem o **Quadro 1** (o custo real de um turno de 5 passos, medido passo a passo), a **Tabela 2** (cada peça: tool/memória/app, papel, onde vive) e explica os datasets e o campo de OpenRouter. **O Quadro 1 é o achado próprio do trabalho** e vale destacar na conversa: a entrada vai de 2,9k a 11,3k tokens dentro de um único turno, contra ~434 de saída, porque cada passo reenvia a conversa inteira mais todas as observações já colhidas. A consequência prática é de projeto: uma tool que devolve JSON cru não custa uma vez, custa uma vez por passo restante do turno. **Por que está aí:** responde as perguntas 8 (o que o protótipo ilustra), 9 (datasets, o que é fake) e 10 (OpenRouter na UI). **Multi-conta (§4.2), e é um bom lugar para mostrar rigor:** o mock tem duas contas, e a segunda (Bravo Pet) é **grupo de controle**, montada como o inverso da primeira. O argumento: *se a resposta for parecida nas duas, o agente não está lendo o dado*. E o desenho por trás: **a conta é estado do harness, nunca argumento de tool**, então o modelo não alcança outro cliente nem por engano nem por instrução plantada no contexto. Se fosse parâmetro, isolamento seria promessa de prompt; sendo estado, é propriedade do runtime, e `build/test_contas.js` falha se alguma tool voltar a expor o campo.

O guard-rail de escopo tem número (§4.1): perguntei ao agente sobre culinária e o turno custou **1 chamada, 0 tools, 2,5k de entrada e 41 de saída (US\$ 0,0004)**, contra 5 chamadas e 39,5k do diagnóstico. Cerca de 16x menos. O ponto não é a recusa, é *onde* ela acontece: antes do loop, então a pergunta fora do domínio não vira investigação nem paga reenvio.

Como defender: "O protótipo não executa a infra do paper. Ele **simula** o resultado: as tools leem um mock cruzado, e o trace mostra o harness orquestrar. O avaliador vê a tese acontecendo."

5. Notas de execução

O que diz: PDF, demo estática, OpenRouter com key só no browser, dados mock determinísticos. **Por que está aí:** deixa claro o que é real e o que é fake, sem inventar métricas.

6. Pontos críticos por tarefa do gestor (a seção que dá credibilidade)

O que diz: percorre as **sete tarefas** do dia a dia (insight da semana, diagnóstico, análise de criativos, brief de criativo, mapa de solução, pauta de call, responder o cliente) e, em cada uma, diz o que o harness faz, onde ele quebra e o que no desenho mitiga, com a coluna final sempre dizendo também **o que ele não resolve**. É a **Tabela 3**. Depois, os dois casos mais graves ganham parágrafo: atribuição (no relatório) e correlação confundida com causa (no diagnóstico). **Por que está aí:** o critério de avaliação pergunta isso com todas as letras ("existem pontos críticos na solução proposta em relação aos tipos de tarefas do dia a dia do gestor?"). Gaps honestos valem pontos: mostram que você conhece os limites.

7. Limitações e próximos passos

O que diz: o que fica de fora por escolha (só-leitura, um cliente, memória mock, política de compactação de contexto em sessão longa), **o que ficou de fora de propósito** (a camada de integrações e a geração de criativo) e "com mais uma semana eu faria X, e deliberadamente NÃO faria Y". **Por que está aí:** responde a pergunta 11.

A parte de integrações é a que mais vale ensaiar, porque é onde a humildade vira argumento em vez de virar buraco. A resposta tem três movimentos, nesta ordem:

1. **O que não existe, sem rodeio.** Nenhuma tela de conectar conta, nenhum OAuth por cliente, nenhum cofre de credencial, nenhum estado de conexão (token expirado, permissão revogada, conta trocada), nenhuma seleção de conta em quem tem várias. E nenhuma geração de imagem ou vídeo: o agente escreve o brief, não produz a peça.
2. **Por quê, com a razão honesta na frente.** Recorte de protótipo é metade. A outra metade é que você **não conhece o escopo completo do harness que a AdzHub pretende**: quem conecta a conta, como a credencial é guardada e rotacionada, como funciona multi-conta, que permissão cada perfil tem, o que acontece se o token cai no meio de um turno. Sem essas respostas, desenhar superfície de credencial é inventar requisito, e requisito inventado em cima de credencial de anúncio custa caro depois. **Deixar o buraco visível é mais defensável que preencher com suposição** — e é exatamente o tipo de decisão que um avaliador de vaga fundacional procura.
3. **Por que isso não derruba a tese.** Pela arquitetura de três camadas, integração nova é **tool nova na allowlist**: o loop não muda. Conectar contas é produto em volta do harness. E geração de criativo entraria como **App (skill)**, não como tool crua, porque carrega metodologia e restrição de marca. Se te perguntarem "e quando plugar o Meta de verdade?", a resposta é: muda a allowlist e o grafo ganha nós, não muda o mecanismo.

⚠ O que NÃO dizer: "não deu tempo". Não é verdade e é mais fraco. O que faltou foi informação de escopo, não hora. **Como defender:** "O join Meta x CRM depende de UTM limpo; no mundo real UTM falta e a atribuição diverge. A hidratação rasa pode não achar o nó certo por sinônimo. O agente correlaciona, não prova causa."

Referências

OpenHands SDK, ReAct, CodeAct, RLM, Zep/Graphiti, Anthropic. São as fontes que sustentam cada decisão. Só citei o que de fato usei.

Onde cada uma das 11 perguntas do roteiro é respondida

#	Pergunta	Onde no paper
1	Qual a tese?	Abstract + §1 (frase de tese)
2	O que deixa de ser chatbot e vira agente?	§1 (modelo vs harness) + §3.2
3	O que você achava e o que mudou?	§1 (segundo parágrafo)
4	Qual abordagem escolheu e por quê vs as outras?	§2.3 + Tabela 1 (veredito das 5) e §3.1 (a composição)
5	Como conversa com supercérebro/Apps/APIs? O que é tool/memória/app?	§2.2 (as naturezas) + §3.2 (o mecanismo)
6	Trade-offs aceitos de propósito	§3.3
7	Onde a solução quebra nas tarefas reais	§6 + Tabela 3 (7 tarefas)
8	O que o protótipo ilustra vs só no paper	§4.1
9	Datasets, o que é fake, o que dá pra testar	§4.2 + Tabela 2
10	Como colar a OPENROUTER_API_KEY e trocar modelo	§4.3
11	Com mais uma semana, o que faria / não faria	§7 (final)

Glossário (para não travar em nenhum termo)

- **Harness:** o runtime em volta do LLM. O loop que executa tools, injeta observações, guarda estado e impõe limites. Não é o modelo.
- **Modelo (LLM):** prevê o próximo token. No máximo emite a intenção de chamar uma função.
- **Tool:** função com assinatura declarada que o modelo pede para chamar; devolve JSON (a observação) que volta ao contexto.
- **Observação:** o resultado de uma tool, reinjetado como nova evidência no loop.
- **Loop ReAct:** ciclo intenção → ação (tool) → observação → intenção, até responder ou bater um limite. (Reasoning + Acting.)
- **Memória como ambiente (RLM):** em vez de o LLM chamar a memória, o harness recupera o contexto relevante e injeta no prompt antes do loop. O agente já começa situado. RLM = Recursive Language Models (contexto vive fora da janela e o agente navega/fatia).
- **Hidratação:** o ato de montar esse "pacote de contexto" (nós do grafo + eventos recentes da timeline) e injetar. No protótipo é uma fase visível no trace.
- **Skill (App):** metodologia encapsulada que o harness invoca; não é reimplementada no prompt. Ex.: análise de criativos devolve ranking + recomendação (seguir/pausar/variá).
- **Allowlist:** conjunto fechado de tools que podem ser chamadas. Guard-rail de harness.
- **max_steps:** teto de passos por turno. Guard-rail de harness. No protótipo é 8.
- **GraphRAG:** RAG (busca aumentada por recuperação) sobre um grafo de conhecimento, não sobre texto solto. É o formato do supercérebro.
- **Supercérebro:** a memória da operação: pessoas, cliente, campanhas, decisões, ligadas em grafo, com linha do tempo (contexto temporal). Estilo Mem0 / Graphiti.

- **CodeAct:** harness onde o agente escreve e executa código num sandbox. Rejeitado aqui.
 - **utm_content:** o parâmetro de URL que casa o anúncio do Meta (`ad_id`) com o lead no CRM. É a chave do "join" que revela o custo por venda real.
-

Perguntas que o avaliador pode fazer (e boas respostas)

"Por que não um grafo de estados, se o domínio já é um grafo?" Porque o supercérebro é um grafo de **dados**, não de **controle**. O chat é aberto: cada mensagem é uma tarefa diferente. Um grafo de controle fixo engessaria. Uso o grafo como memória.

"Por que não deixar tudo como tool e simplificar?" Porque memória e metodologia têm custo se viram tool. A memória vira adivinhação (o LLM esquece de buscar), e a metodologia vira reimplementação no prompt (cara e divergente). Cada natureza pede um mecanismo.

"Seu protótipo não executa a infra de verdade. Isso não enfraquece?" Não, porque o desafio pede para **simular/demonstrar o resultado**. O que prova a tese é o trace: o harness hidrata memória, chama tools de API e invoca skills, e a resposta é ancorada nos números do mock, não num palpite. Os 5 problemas plantados são descobertos, não estão escritos no dado.

"Onde isso quebra num cliente real?" UTM sujo quebra o join; atribuição multi-toque diverge entre Meta e CRM; a hidratação rasa (sem embeddings) pode errar o nó por sinônimo; e o agente correlaciona, não prova causa. Está tudo em §6.

"E segurança? O agente pode pausar um anúncio errado?" Não. Por decisão de arquitetura o MVP é **só-leitura**. Nenhuma tool escreve na conta. O agente propõe, o humano executa. Isso remove a classe inteira de risco de ação indevida.

"Com mais tempo, o que faria primeiro?" Recuperação de memória de verdade (grafo temporal com embeddings e decaimento, estilo Graphiti/Zep), para a hidratação parar de depender de match por substring.

Os 5 problemas plantados no dataset (o que a demo prova)

O mock foi construído para esconder 5 problemas de gestão que o agente descobre cruzando fontes. Nenhum está rotulado no dado.

1. **Criativo caro vs barato:** o vídeo de depoimento gasta R\$4.200 para 2 vendas (CPA R\$2.100), enquanto o "antes/depois" gasta R\$900 para 9 vendas (CPA R\$100). Só aparece cruzando Meta x CRM.
2. **Criativo saturado:** o `hook_rate` do depoimento cai de 32% para 18% e a frequência sobe de 1,8 para 4,8. O App recomenda pausar.
3. **Origem inconsistente:** 12 leads dizem "Google", mas só 4 têm UTM de Google; os outros vieram do Meta (um UGC de Whey). Decidir verba pela origem declarada erraria o alvo.
4. **Aprovação travada:** a campanha de Creatina está parada porque a peça diz "resultado garantido", que o mapa de solução lista como proibido de falar. Só se descobre lendo timeline + WhatsApp + mapa.
5. **Spend vs budget:** o Ômega 3 gastou R\$6.200 contra um teto de R\$5.000, puxado pelo mesmo depoimento. Uma ação (pausar o depoimento) resolve os problemas 1, 2 e 5 de uma vez.

O ponto que impressiona: **um único achado (pausar o depoimento) costura três sintomas**. Isso é o que separa um agente de um chatbot que responde uma pergunta por vez.

Como editar o paper (mecânica)

A fonte é `paper/paper.html`, HTML puro com o layout de duas colunas em CSS. O PDF é gerado por um comando só, sem LaTeX e sem dependência:

```
`` chromium --headless --disable-gpu --no-sandbox \ --print-to-pdf=paper/paper.pdf --no-pdf-
header-footer "file://$PWD/paper/paper.html" ``
```

Três coisas que vale saber antes de mexer:

- **Onde escrever.** Texto corrido vai em `<p>`, seção em `<h2>`, subseção em `<h3>`. O corpo inteiro vive dentro de `<div class="paper">`, que é o que cria as duas colunas.
- **Elemento largo atravessa as colunas** com a classe `span-all` (é o caso da Figura 1, do Quadro 1 e das três tabelas). Sem ela, uma tabela larga fica espremida em meia página.
- **Legenda e tabela precisam ficar juntas.** Tabela dentro de `<div class="bloco-tabela">` não se separa da legenda na quebra de página. Isso não é preciosismo: sem o bloco, a legenda da Tabela 1 ficou órfã no fim de uma página e a tabela apareceu sozinha na seguinte.

Depois de gerar, confira o resultado **como página**, não como código: 6 páginas, abaixo de 3 MB, e nenhuma tabela cortada no meio. Um jeito rápido de olhar página a página:

```
`` python3 -c "import pymupdf; d=pymupdf.open('paper/paper.pdf'); \
[p.get_pixmap(dpi=105).save(f'pg{i+1}.png') for i,p in enumerate(d)]" ``
```

Pendente

Nada no paper. Os três assuntos que estavam abertos (contexto em sessão longa, avaliação do harness, falha de tool) foram escritos: os dois primeiros viraram §3.4 e §3.5, e o multi-conta virou código.